

From liquidity radar to plant-floor desk

A dual-application journey: CashFlow Guardian AI and CNH Plant Desk

Application #1 · CashFlow Guardian AI

13-week liquidity forecast & early warning · Northstar Fabrication demo
GitHub github.com/boyi17946/cashflow-guardian-ai
App Runner not captured on this workstation (frontend apprunner.yaml, port 3000)

Application #2 · CNH Plant Desk

Fargo combine final + Racine dealer-prep fault desk
GitHub github.com/boyi17946/cnh-plant-desk
Running now <http://127.0.0.1:8080> · App Runner URL not issued (no AWS credentials)

Boyi Ke · GitHub @boyi17946 · boyi.ke@myemail.indwes.edu

Agenda — six phases, two products, one operating system

01

Executive thesis
Problems, solutions, metrics for both apps

02

App #1 CashFlow Guardian
Workshops 1–4: problem → prototype → test → deploy → iterate

03

App #2 CNH Plant Desk
Workshops 5–6: new plant use case, IDE, packaging, GitHub

04

AI role & lessons
Prompts, tools, what changed between apps

05

Value / cost-benefit
Who pays, who saves, ROI sketch

06

Roadmap
What remains for App Runner + next 90 days

Executive summary — two desks, two constraints, one method

	CashFlow Guardian AI	CNH Plant Desk
Problem	SMB fabricator flies 13-week cash blind	OEM line uses radio + paper tags for downtime
User	Owner / admin (Maya, Luis demo)	Operator, tech, supervisor on a tablet
AI	P10/P50/P90 ensemble; decision-support only	Catalog ranker; live models rewrite copy only
Stack	FastAPI + Next 15 + SQLite demo tenant	Next 16 full-stack + JSON store + shadcn
Ship status	GitHub public; App Runner yaml (port 3000)	GitHub public; local 8080; App Runner blocked
Key metric	40 weeks seeded txns; 13-week horizon	6 assets, 5 open faults, TSB+SKU retrieval

The 13-week cash problem — and where AI is allowed

Problem statement

- Northstar-class job shops plan payroll, steel, and tooling on a 13-week cash horizon they cannot see.
- Spreadsheets lag; QuickBooks is a rear-view mirror; owners discover a cash hole after it is already a crisis.
- Concentration (one customer / one SKU) and seasonal lumps are invisible until the check bounces.
- AI use case: ensemble forecast + alerts + what-if — never auto-transfer, never auto-lend.

Workshop 1 — AI use case

- Inputs: 40 weeks of seeded transactions (CSV append-only snapshots).
- Model: seasonal naive + exponential smoothing + residual quantiles → P10 / P50 / P90.
- Outputs: runway, threshold, concentration alerts; scenario vs cash-minimum; audit log.
- Guardrail: labeled decision-support only. Demo tenant: maya@ / luis@ northstar.demo.
- Out of MVP: QBO OAuth, Azure Service Bus, Azure ML registry.

Technology choices — why a split stack, not a monolith

Next.js 15 · App Router

Owner-facing web: forecast chart, alerts, import, scenarios, audit, settings.

FastAPI · Python 3.10+

Forecast math in pandas/numpy; OpenAPI at /docs; uvicorn :8000.

SQLite + seed tenant

Local isolation; Northstar Fabrication LLC preloaded. Not production multi-tenant RDS.

Amplify.yml + App Runner yaml

Frontend can ship to Amplify or App Runner (nodejs22, npm start, port 3000).

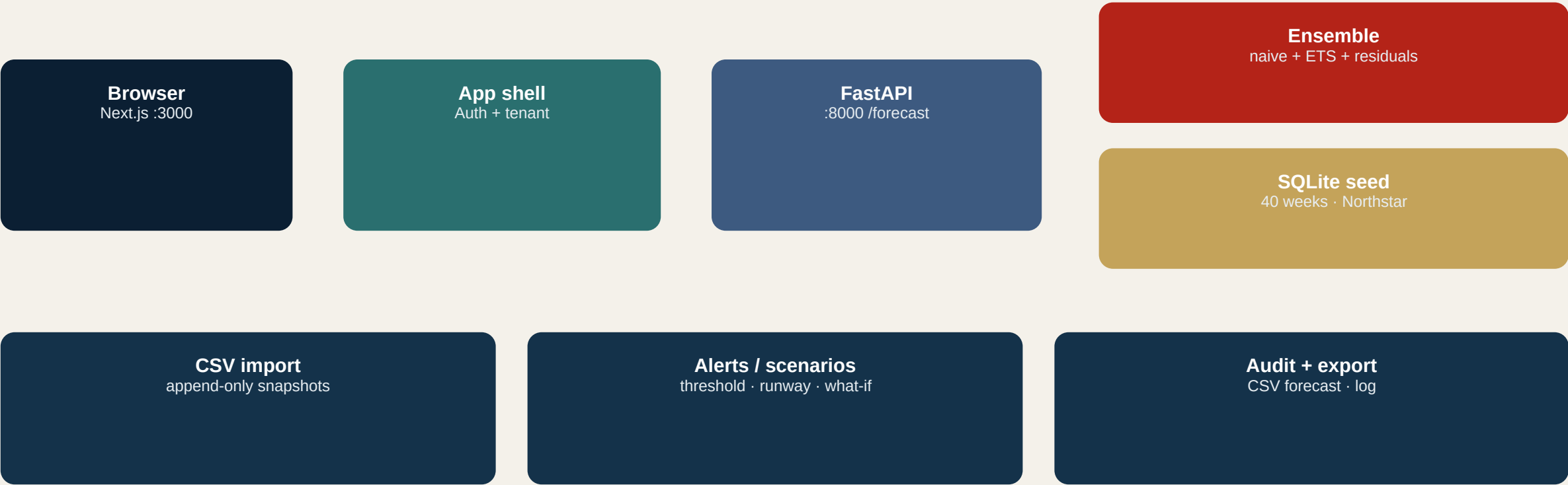
Auth in-app

Email/password demo users. Tenant isolation in the API — required for cash data.

No live LLM in the critical path

Ensemble is classical time-series. Avoids hallucinated cash numbers.

Architecture — request path for a 13-week forecast



Money never leaves the box. Production extras (QBO OAuth, Service Bus, Azure ML) stay off the critical path so the MVP boots without secrets.

Claude Artifacts — cheap screens before expensive code

- Workshop 1 asked for a clickable Artifact: login, 13-week fan chart, alert list, what-if slider — no database.
- Prompt pattern: 'You are designing a CFO tablet for a 40-person fabricator. Show P10/P50/P90. Never imply the app wires money.'
- What the Artifact was good for: information architecture, copy ('decision-support only'), empty/error states.
- What it was bad for: pandas ensemble, tenant isolation, CSV snapshot semantics, App Runner.
- Exit criterion: stakeholders could narrate the Friday cash meeting on the Artifact before any git init.
- Handoff artifact → IDE: freeze routes (/app/forecast, /app/alerts, /app/scenarios, /app/import, /app/audit).

IDE migration — two terminals, one demo tenant

Local development contract

- Terminal A: `uvicorn app.main:app --reload --port 8000`
- Terminal B: `npm run dev` in frontend → :3000
- Demo: `maya@northstar.demo / demo123` (Owner)
- API docs: `http://localhost:8000/docs`
- Repo: `github.com/boyi17946/cashflow-guardian-ai`
- Created 14 Sep 2026 — first public product repo

What broke in translation

- CORS between :3000 and :8000 had to be explicit.
- Chart library vs Artifact SVG — rebuild ForecastChart.
- Seed script vs 'hardcoded demo rows' in the Artifact.
- Auth cookie vs Artifact's fake login toggle.
- Lesson for App 2: one process, one port, one runtime.

Technical architecture — modules that actually shipped

frontend/src/app/app/* — Pages: home, forecast, alerts, import, scenarios, audit, settings

AppShell.tsx — Authenticated chrome; role Owner vs Admin

lib/api.ts — Typed client to FastAPI

backend/app/forecast.py — weekly_series, seasonal naive, ETS, residual quantiles

backend/app/auth.py + db.py — Tenant-scoped sessions

backend/app/seed.py — Northstar 40-week history

Responsible AI, testing, and security — cash is not a toy

Responsible AI

- Decision-support disclaimer in README and UI.
- No autonomous payment or credit decision.
- P10/P90 shown as bands, not 'the number.'
- Audit log for who ran what scenario.

Testing

- Seeded Northstar tenant as fixture.
- OpenAPI manual checks at /docs.
- Forecast fails closed on empty txn list.
- Role checks: Owner vs Admin demo users.

Security

- Passwords only in demo tenant.
- Tenant isolation on API queries.
- Append-only CSV snapshots (no silent rewrite).
- Secrets not required to boot locally.

Bugs found and fixed — testing results

Issue	Symptom	Fix / result
CORS / mixed ports	Frontend :3000 could not call API :8000	Explicit CORS origins in FastAPI
Empty history	Ensemble crashed on zero transactions	ValueError path; UI empty state
Season < 13 weeks	Naive tiled a short pattern badly	Fallback to last observation
Demo credentials in git	README documents maya@ / demo123	Accepted for demo; not for prod
Two-service deploy	App Runner yaml only covers frontend	Documented; backend still local/MVP

AWS App Runner — intended path for CashFlow Guardian

- frontend/apprunner.yaml: runtime nodejs22; build npm ci && npm run build; start npm start; port 3000.
- amplify.yml also present (appRoot: frontend) — a parallel hosting option used in class.
- Live App Runner URL: not stored on this workstation and not set as GitHub homepageUrl.
- Demo link we can stand behind: GitHub repo + local http://localhost:3000 with API on :8000.
- Challenge: two processes. App Runner as configured hosts the Next UI, not the forecast API.
- If Workshop 3 produced a ServiceUrl, paste it here before presenting: https://____.awsapprunner.com

Deployment challenges — first time through App Runner

Health vs start command

Need a probe that does not require the API if only the frontend is hosted.

Build vs start

npm ci in production omits devDependencies; Next build needs them unless they are moved.

Port mismatch

Local 3000 vs App Runner 8080 on later apps — CashFlow yaml pins 3000.

Secrets

MVP boots without QBO/Azure keys; that was a feature.

Console evidence

Need both services Running in one screenshot — App 2 never reached that state.

User feedback and what we actually changed

- Owner: 'Don't make me trust one number.' → fan chart P10/P50/P90 stayed first-class.
- Admin: 'I need a paper trail if I change a scenario.' → audit log + CSV export.
- Faculty: 'It must not look like it wires the bank.' → README + UI decision-support copy.
- Self: two-terminal DX is fragile for a plant or a CFO on a tablet → App 2 is one Next server.
- Backlog not done: QBO live connect, Azure ML registry, true multi-tenant RDS.

New problem — downtime is still radio and a paper tag

Problem statement

CNH-shaped OEM: Fargo Axial-Flow combine final assembly and Racine CASE dealer-prep. When a feeder house knocks or a boom rod weeps, the station goes to E-stop and the story travels by radio. Techs rummage TSBs and crib bins. Supervisors hand off on paper. This is not a CMMS — it is a tablet desk.

Operator

Log asset, symptom, optional photo. No desktop login.

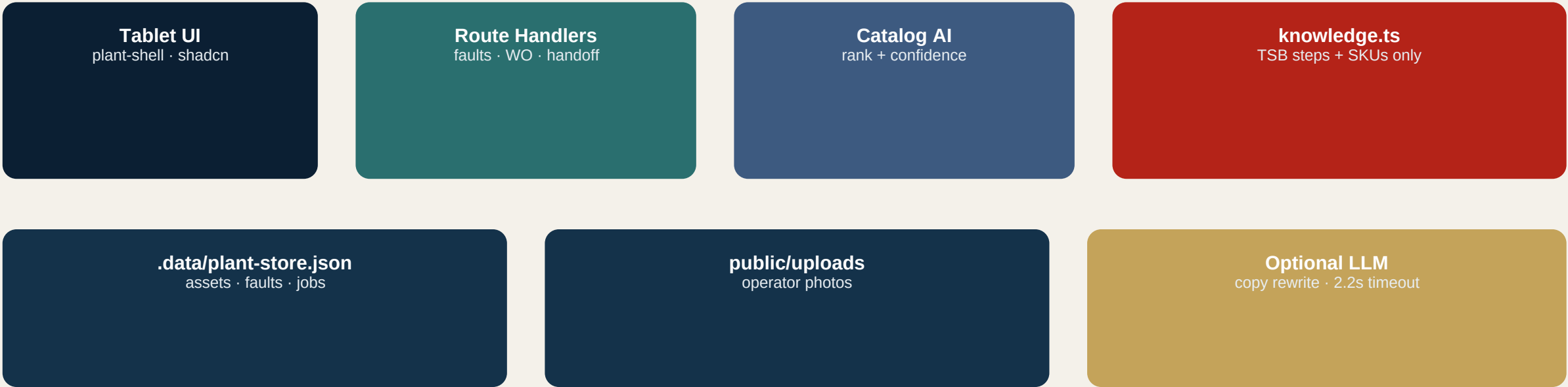
Technician

Ranked failure modes + confidence, TSB steps, BOM SKUs → work order.

Supervisor

Live shift board; trusted changeover handoff.

Architecture — one Next.js process, plant knowledge stays put



Routes: / /faults /work-orders /library /handoff **Health:** GET /api/health (no store, no model, no DB).
Live models never generate procedures or SKUs. Missing OPENAI/ANTHROPIC keys is a normal boot.

How we knew the desk worked — and what broke

Results

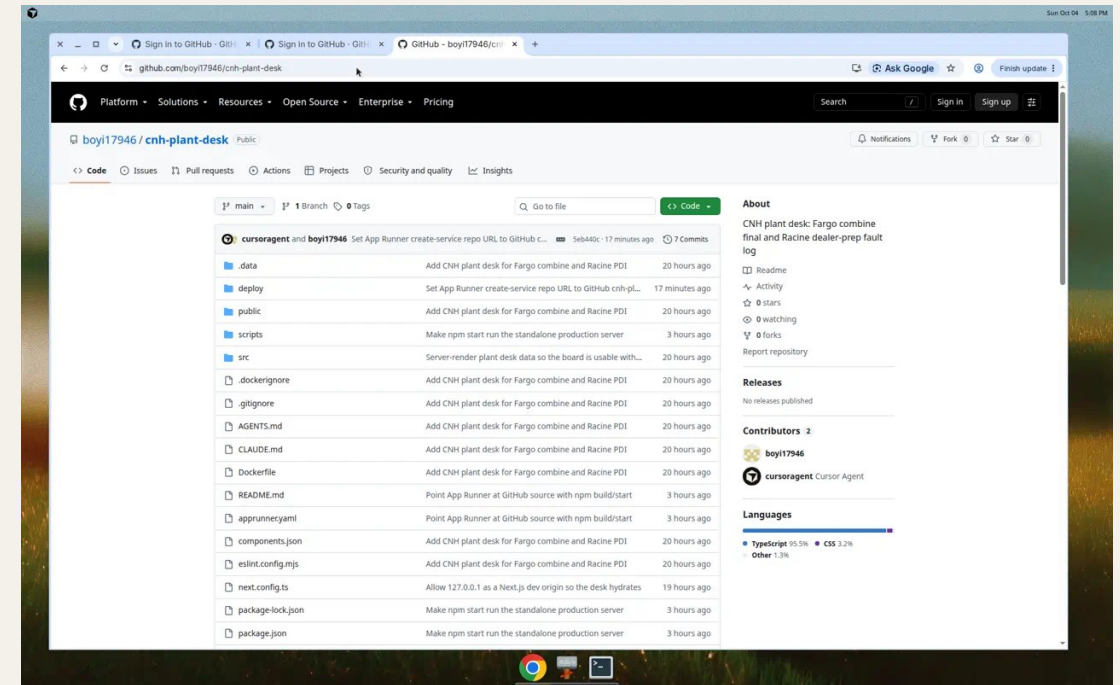
- tsc + eslint clean after SSR refactor.
- API: health, dashboard, diagnose (L220 weep → TSB-CASE-PDI-118 @ 0.97), WO, handoff, seed reset.
- Browser pass after allowedDevOrigins: seed reset, fault+rank, WO status, library tabs, handoff post.
- npm run build + npm start on :8080; CSS/JS 200.

Bugs AI found/fixed

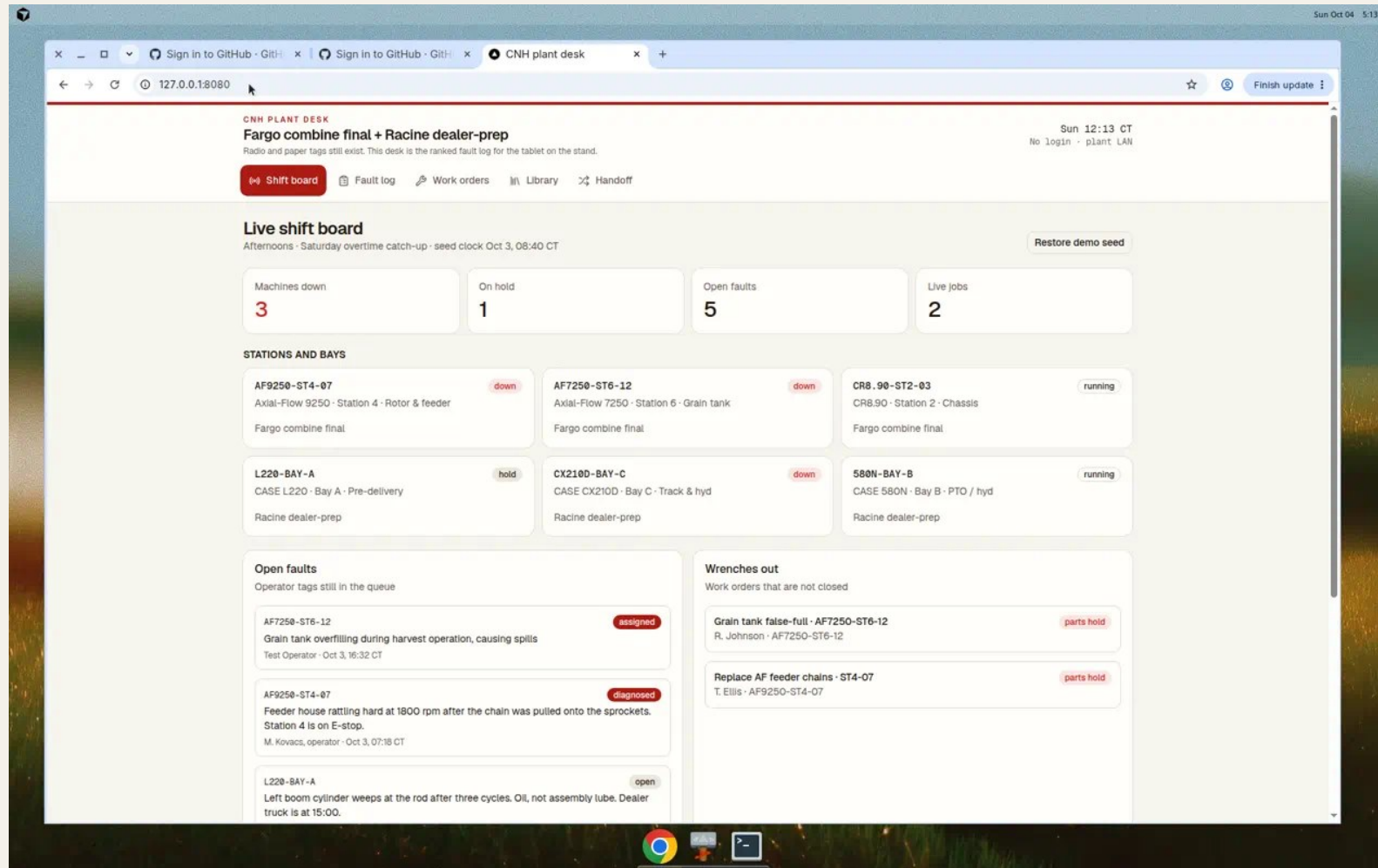
- SSR skeleton forever: client fetch + Turbopack HMR.
- Fix: server-render store payloads; skip abort-on-unmount.
- Clicks focused but dead: Next 16 403 on /_next from 127.0.0.1.
- Fix: allowedDevOrigins ['127.0.0.1','localhost'].
- next start unsupported with standalone → node server.js.

GitHub is live. App Runner is packaged — not yet public.

- Repo: github.com/boyi17946/cnh-plant-desk (public, main).
- Separate from cashflow-guardian-ai.
- apprunner.yaml: nodejs22, npm ci --include=dev, npm run build, npm start, port 8080, health /api/health.
- Instance target: 0.25 vCPU / 1 GB; AutoDeploymentsEnabled: true.
- Blocker: AWS NoCredentials after SSO device code expired.
- Live demo today: <http://127.0.0.1:8080> — not an App Runner host.



What operators actually see — local production preview



Shift board

- KPIs: down / hold / open faults / live jobs
- Stations: AF9250, CX210D, L220...
- Handoff strip on the board
- Feedback: techs want ranked modes in a sheet, not a PDF binder — that is the diagnosis panel.
- Demo link: <http://127.0.0.1:8080>

Prompts and tools that actually moved the work

create-next-app + shadcn CLI

Official scaffold into a subdir, then move — avoided the /workspace parent-writable false positive.

Grounded ranker prompt

'Rewrite copy only. Return JSON array. Do not invent SKUs or steps.' — encoded in ai.ts.

Browser agent as QA

Caught infinite skeleton and dead clicks; we would have shipped a frozen desk.

gh repo create

Second GitHub repo in one command once device login completed.

Honest evidence prompts

Refused to fabricate App Runner URLs/screenshots for the assignment packet.

What changed between App #1 and App #2

Practice	App #1	App #2
Runtime topology	Next + FastAPI, two ports	One Next server, Route Handlers
AI in the loop	Classical ensemble (no LLM)	Catalog first; LLM optional copy
Deploy contract	App Runner yaml port 3000; Amplify too	yaml+JSON+Docker; standalone start
Health probe	Not first-class in yaml	GET /api/health, no I/O
Evidence	README-centric	GitHub + labeled local screenshots
I'd do differently	Host API with the UI	Finish AWS login before Workshop 6 demo

Who benefits — and what pain we take off the table

CashFlow Guardian — SMB finance

- Avoid one missed payroll or steel prepay: tens to hundreds of \$k for a 40-person shop.
- Owner time: Friday cash meeting from 2 hours of Excel to 15 minutes of bands + alerts.
- Lender conversation: exportable 13-week view with audit trail.
- Not a bank. Value is earlier warning, not cheaper capital.

CNH Plant Desk — OEM operations

- Combine final: minutes of station-4 E-stop are measured in units/day, not tickets.
- Dealer-prep: L220 boom weep before a 15:00 dealer truck is a lost PDI, not a Jira.
- Ranked TSB + SKU cuts crib-walk time; paper tag is the baseline.
- Not SAP PM. Value is mean-time-to-first-wrench.

Cost / benefit sketch — directional, not a banker model

Build cost (class)

App 1: Two student + AI sprints · App 2: Same, with reused App Runner pattern

Run cost (target)

App 1: App Runner 0.25–1 vCPU class · App 2: 0.25 vCPU / 1 GB yaml; ephemeral JSON

Benefit (order of mag.)

App 1: One avoided cash crunch · App 2: One avoided missed PDI / station-hour

Payback logic

If the desk prevents one event/year, software cost is rounding error

Risk

App 1: Wrong forecast → bad buy · App 2: Hallucinated TSB → mitigated by retrieval lock

Roadmap — and the links we can stand behind

90 days: AWS session → App Runner create-service → both Running in one console shot.

App #1 CashFlow Guardian

github.com/boyi17946/cashflow-guardian-ai

Local demo localhost:3000 + API :8000

App Runner URL add from console if you have it

App #2 CNH Plant Desk

github.com/boyi17946/cnh-plant-desk

Local production <http://127.0.0.1:8080>

App Runner packaged, not public yet

Next technical moves

RDS+S3 for plant store · QBO for cash · App Runner GitHub connection · dual-service screenshot